

LAPORAN PRAKTIKUM SISTEM OPERASI

MODUL 11: MEMORI XINU



Disusun Oleh :

Nama : Ganes Gemi Putra

Kelas : SE-07-02

NIM : 2311104075

Dosen : Yudha Islami Sulistya S.Kom M.Cs

PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

BAB I PENDAHULUAN

1.1 Tujuan Praktikum

Tujuan praktikum Modul 11 adalah memahami struktur memori pada Xinu dan menerapkan pembuatan command baru pada shell Xinu untuk menampilkan free memory list serta menghitung total free space.

1.2 Dasar Teori Singkat

Pada Xinu, memory image dibagi menjadi text segment, data segment, BSS segment, dan free space. Text segment berisi instruksi hasil kompilasi, data segment berisi variabel global yang sudah diinisialisasi, BSS segment berisi variabel global yang belum diinisialisasi, sedangkan free space digunakan untuk alokasi memori dinamis.

Free space dapat digunakan untuk stack dan heap. Stack dialokasikan ketika fungsi atau proses membutuhkan ruang eksekusi dan akan dilepas ketika fungsi atau proses selesai. Heap digunakan untuk alokasi memori dinamis sesuai permintaan, sehingga harus dikelola dengan lebih hati-hati karena dapat menimbulkan fragmentasi dan memory leak.

Xinu menyimpan informasi free block memory menggunakan linked list melalui variabel global memlist. Setiap block menyimpan alamat block, panjang block, dan pointer menuju block berikutnya. Mekanisme ini memudahkan proses pencarian, alokasi, dan pembaruan daftar memory kosong.

BAB II ALAT DAN BAHAN

- Oracle VM VirtualBox dengan environment Xinu.
- Terminal Linux dan editor gedit.
- Source code Xinu pada direktori ~/xinu.
- File xsh_memstat.c sebagai referensi fungsi printFreeList.
- File include/prototypes.h dan shell/shell.c untuk mendaftarkan command baru.

BAB III LANGKAH KERJA DAN HASIL

3.1 Membuat Command Baru freememory

Langkah pertama dilakukan dengan masuk ke direktori shell Xinu, kemudian menyalin file xsh_memstat.c menjadi xsh_freememory.c. File baru tersebut diedit menggunakan gedit agar nama fungsi command berubah menjadi xsh_freememory dan output disesuaikan dengan kebutuhan jurnal.

```
cd ~/xinu/shell
cp xsh_memstat.c xsh_freememory.c
gedit xsh_freememory.c
```

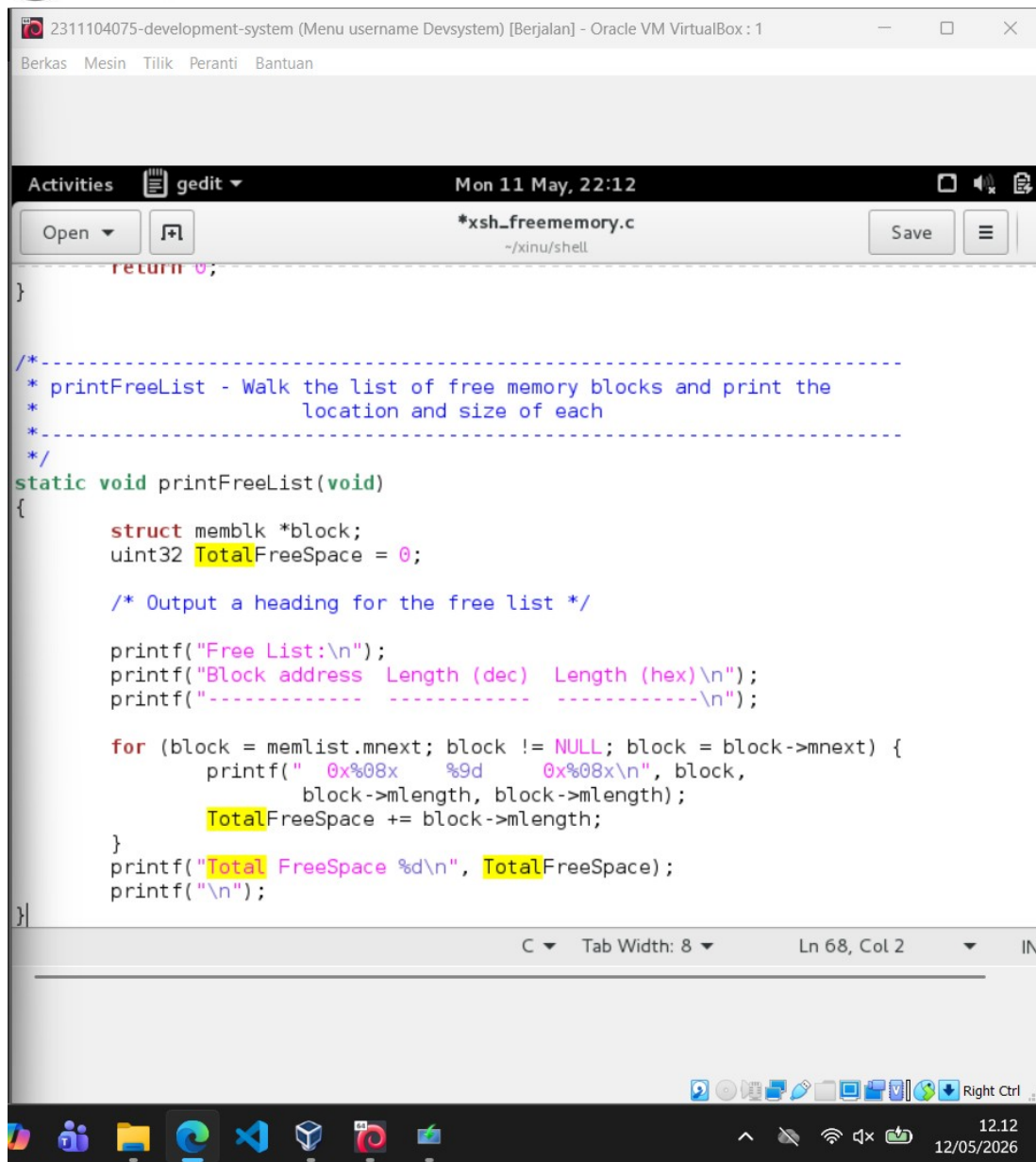
Pada fungsi `printFreeList` ditambahkan variabel `TotalFreeSpace` untuk menampung total seluruh ukuran free block. Nilai `block->mlength` dijumlahkan di dalam loop, kemudian totalnya ditampilkan di akhir output.

```
static void printFreeList(void)
{
    struct memblk *block;
    uint32 TotalFreeSpace = 0;

    printf("Free List:\n");
    printf("Block address   Length (dec)   Length (hex)\n");
    printf("-----\n");

    for (block = memlist.mnext; block != NULL; block = block->mnext) {
        printf("  0x%08x  %9d  0x%08x\n", block,
               block->mlength, block->mlength);
        TotalFreeSpace += block->mlength;
    }

    printf("Total FreeSpace %d\n", TotalFreeSpace);
    printf("\n");
}
```



```

}

/*-----
 * printFreeList - Walk the list of free memory blocks and print the
 *                  location and size of each
 *-----
 */
static void printFreeList(void)
{
    struct memblk *block;
    uint32 TotalFreeSpace = 0;

    /* Output a heading for the free list */

    printf("Free List:\n");
    printf("Block address  Length (dec)  Length (hex)\n");
    printf("-----\n");

    for (block = memlist.mnext; block != NULL; block = block->mnext) {
        printf("  0x%08x    %9d    0x%08x\n", block,
               block->mlength, block->mlength);
        TotalFreeSpace += block->mlength;
    }
    printf("Total FreeSpace %d\n", TotalFreeSpace);
    printf("\n");
}

```

Gambar 1. Source code xsh_freememory.c bagian fungsi printFreeList

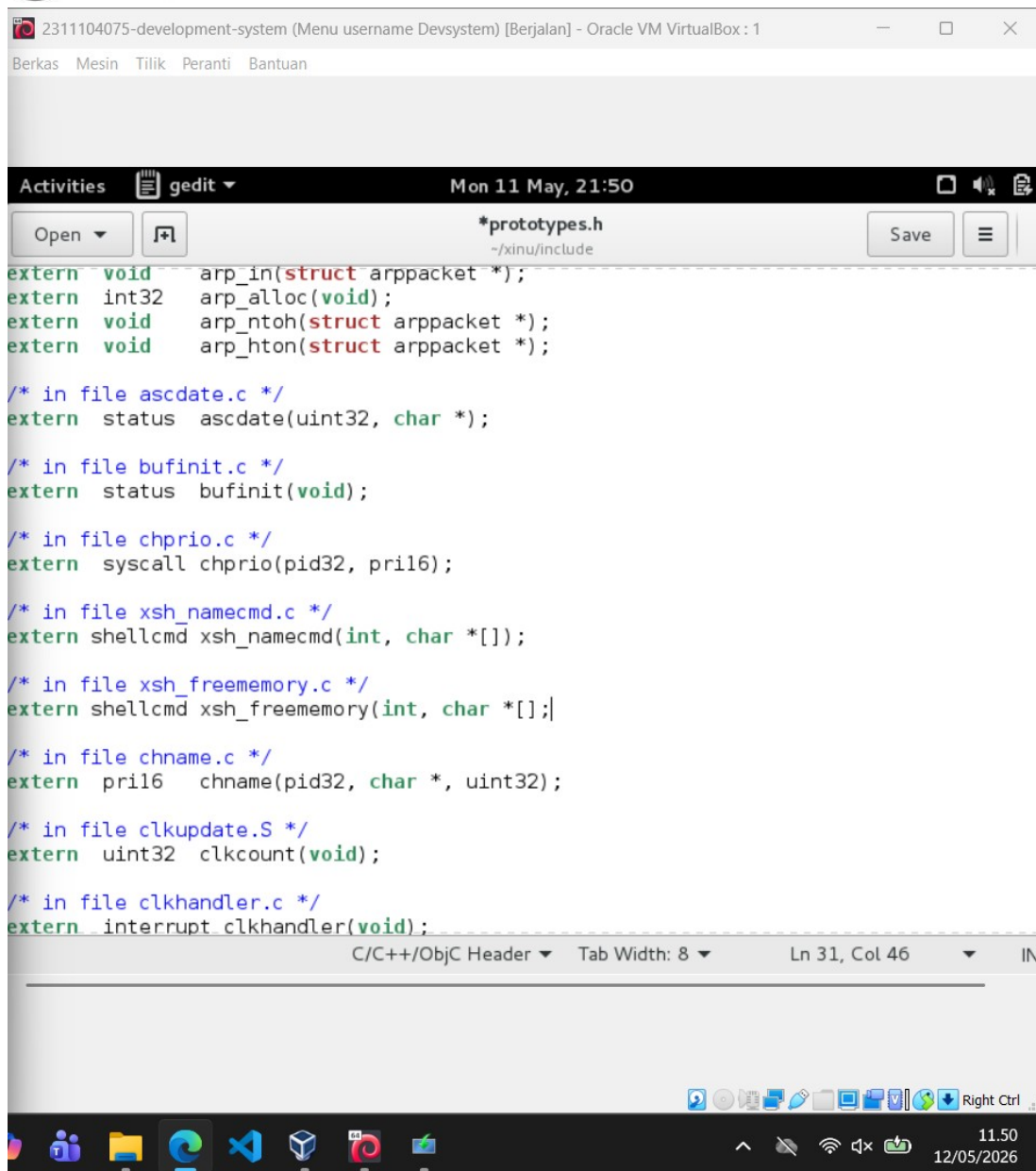
3.2 Menambahkan Prototype Command

Agar fungsi command dikenali oleh proses kompilasi, prototype xsh_freememory ditambahkan pada file include/prototypes.h.

```

/* in file xsh_freememory.c */
extern shellcmd xsh_freememory(int, char *[]);

```



```

extern void    arp_in(struct arppacket *);
extern int32   arp_alloc(void);
extern void    arp_ntoh(struct arppacket *);
extern void    arp_hton(struct arppacket *);

/* in file ascdate.c */
extern status  ascdate(uint32, char *);

/* in file bufinit.c */
extern status  bufinit(void);

/* in file chprio.c */
extern syscall chprio(pid32, pri16);

/* in file xsh_namecmd.c */
extern shellcmd xsh_namecmd(int, char *[]);

/* in file xsh_freememory.c */
extern shellcmd xsh_freememory(int, char *[]);

/* in file chname.c */
extern pri16    chname(pid32, char *, uint32);

/* in file clkupdate.S */
extern uint32   clkcount(void);

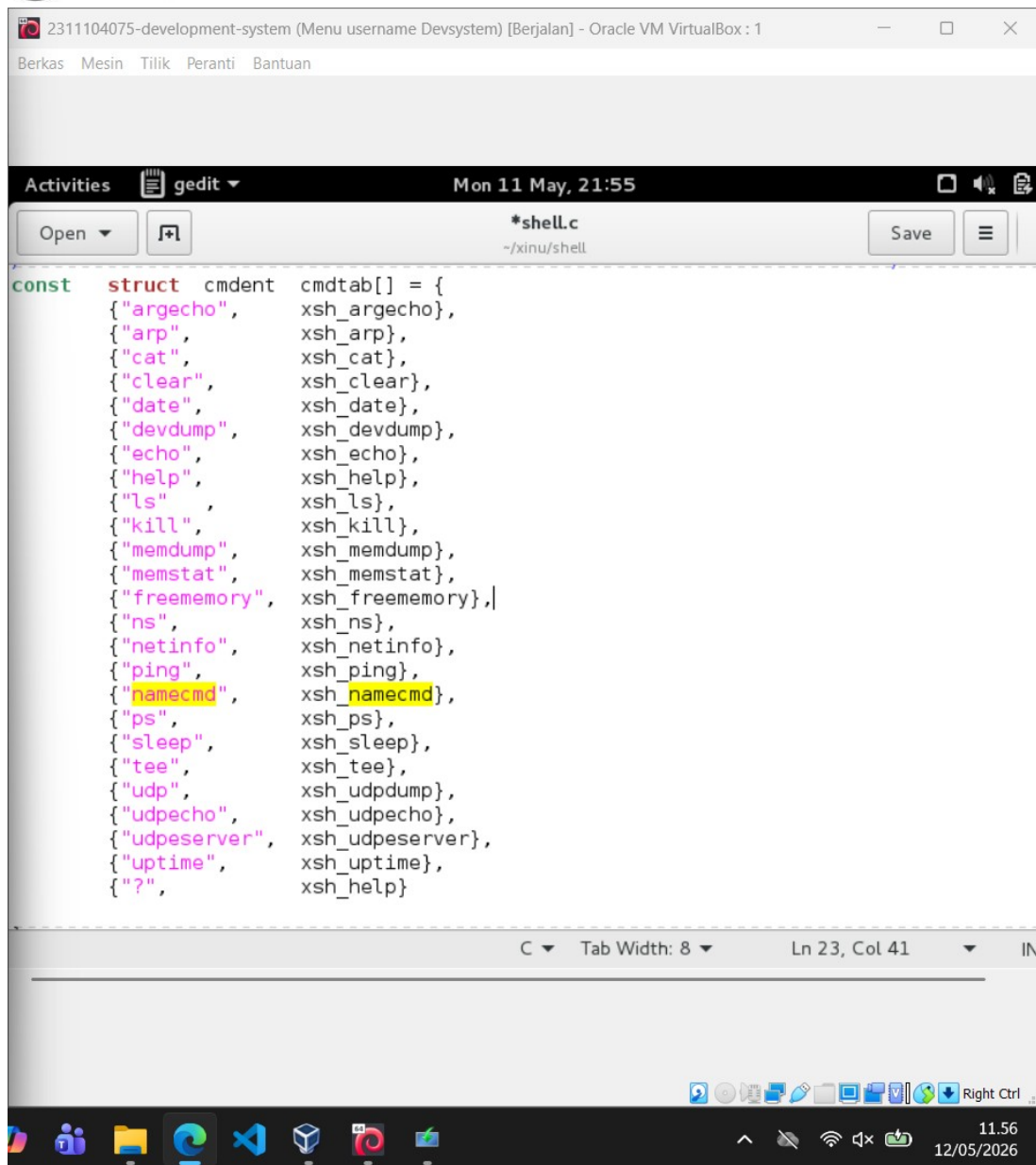
/* in file clkhandler.c */
extern interrupt clkhandler(void);
  
```

Gambar 2. Penambahan prototype xsh_freememory pada prototypes.h

3.3 Mendaftarkan Command pada Shell

Command freememory didaftarkan pada tabel command di file shell/shell.c agar dapat dipanggil langsung dari prompt xsh.

```
{"freememory", xsh_freememory},
```

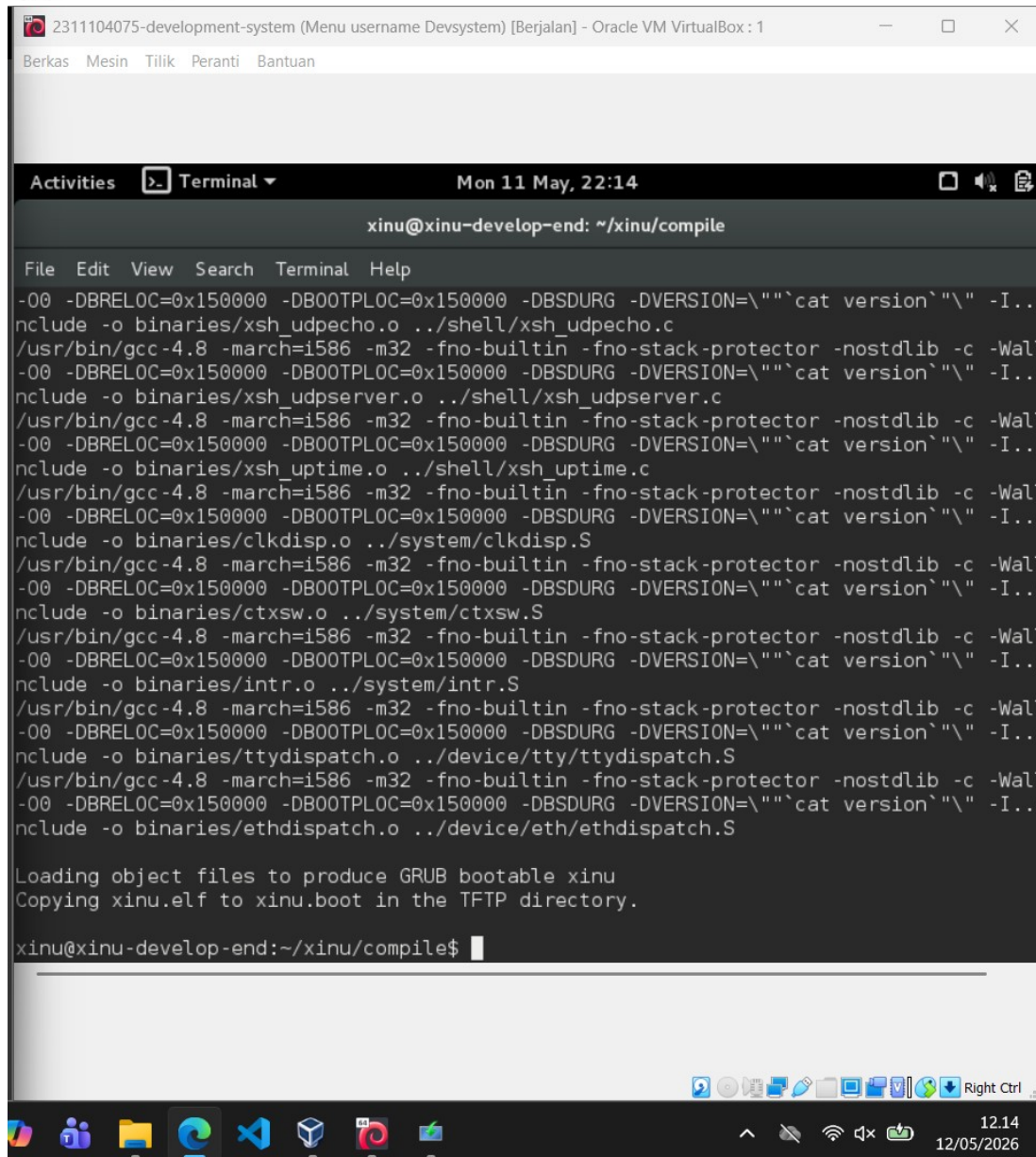


Gambar 3. Pendaftaran command freememory pada shell.c

3.4 Kompilasi Program

Setelah source code dan tabel command selesai diperbarui, Xinu dikompilasi ulang dari direktori compile menggunakan make clean dan make.

```
cd ~/xinu/compile
make clean
make
```



```

xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$

```

Gambar 4. Proses kompilasi Xinu berhasil

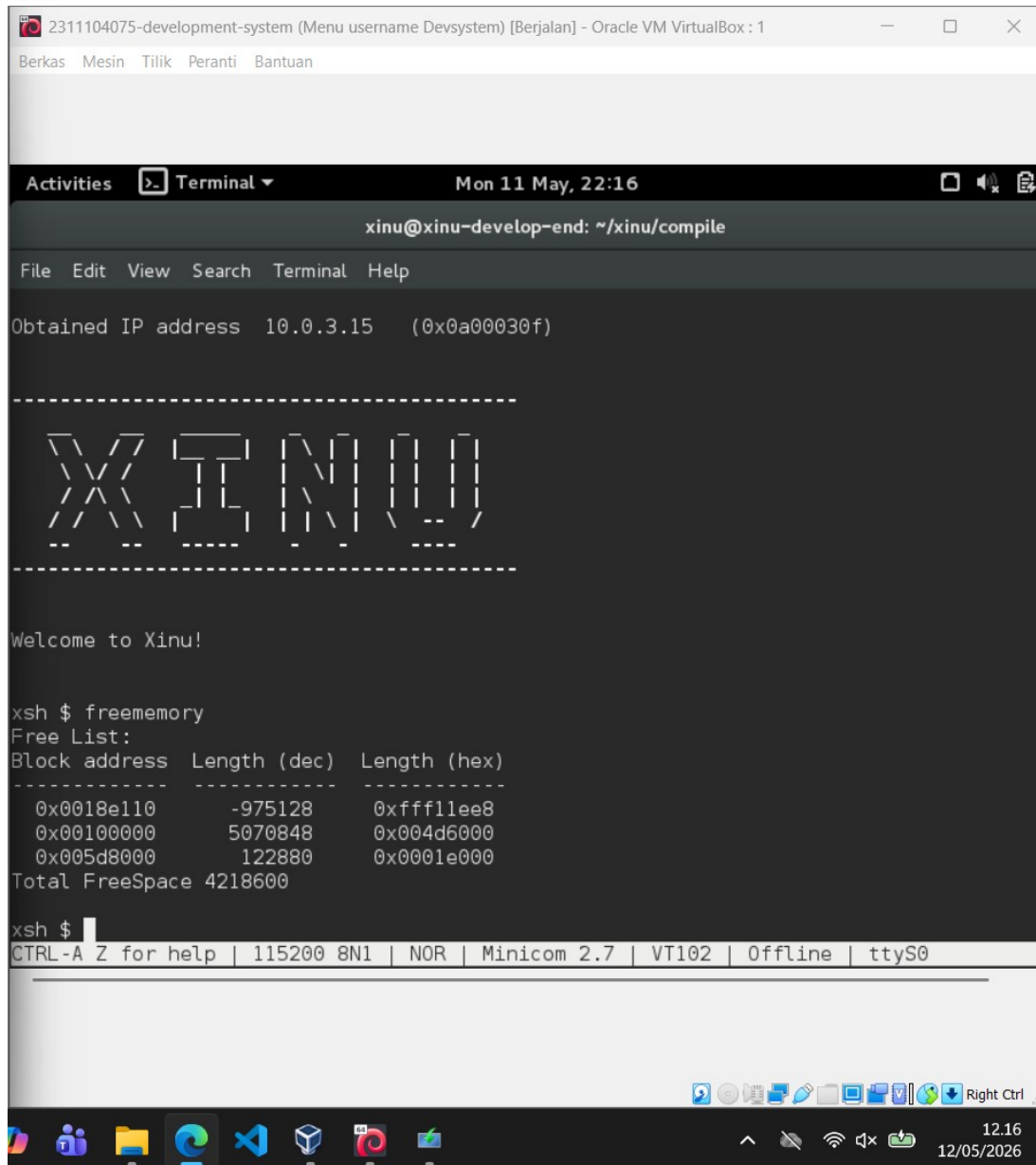
3.5 Menjalankan Command freememory

Setelah kompilasi berhasil, Xinu dijalankan melalui minicom. Command freememory kemudian dieksekusi pada shell Xinu dan berhasil menampilkan daftar free memory block beserta total free space.

```

sudo minicom
freememory

```

```

xinu@xinu-develop-end: ~/xinu/compile
Obtained IP address 10.0.3.15 (0x0a00030f)

-----
XINU
-----

Welcome to Xinu!

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018e110      -975128      0xffff11ee8
0x00100000       5070848      0x004d6000
0x005d8000       122880       0x0001e000
Total FreeSpace 4218600

xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0

```

Gambar 5. Output command freememory pada shell Xinu

Berdasarkan hasil running, command freememory berhasil menampilkan tiga block free memory dan total free space sebesar 4218600 byte. Nilai tersebut merupakan hasil penjumlahan dari seluruh field mlength pada linked list memlist.

BAB IV JAWABAN SOAL NO. 2

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Xinu memisahkan data segment dan BSS segment karena keduanya menyimpan jenis variabel global yang berbeda. Data segment berisi variabel global yang sudah memiliki nilai

awal, sedangkan BSS segment berisi variabel global yang belum diinisialisasi. Pemisahan ini membuat proses inisialisasi memori saat booting menjadi lebih teratur dan efisien.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Ketika proses baru dibuat atau heap dialokasikan, sebagian free space akan digunakan sehingga ukurannya berkurang. Sebaliknya, ketika proses selesai atau memori dilepaskan, ruang tersebut akan dikembalikan ke free space. Karena alokasi dan dealokasi terjadi selama sistem berjalan, ukuran free space bersifat dinamis.

c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Heap lebih berpotensi menimbulkan masalah karena alokasi dan dealokasinya dilakukan secara manual. Jika heap tidak dibebaskan secara eksplisit, dapat terjadi memory leak. Selain itu, heap dapat mengalami fragmentasi dan permintaan heap bisa melebihi free space yang tersedia.

d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Xinu menggunakan linked list karena free block memory dapat tersebar pada alamat yang berbeda-beda. Dengan linked list, Xinu dapat menyimpan alamat awal block, ukuran block, dan pointer ke block berikutnya. Struktur ini memudahkan pencarian block kosong, proses alokasi, dealokasi, dan update daftar free memory.

e. Apa tantangan utama dalam penggunaan heap di Xinu?

Tantangan utama dalam penggunaan heap di Xinu adalah memastikan memori yang telah dialokasikan dapat dibebaskan kembali secara manual. Jika tidak, akan terjadi memory leak. Tantangan lainnya adalah mencegah fragmentasi dan memastikan permintaan heap tidak melebihi kapasitas free space yang tersedia.

BAB V KESIMPULAN

Pada praktikum Modul 11, dapat disimpulkan bahwa Xinu mengelola memori melalui pembagian segment seperti text, data, BSS, dan free space. Free space digunakan untuk alokasi dinamis berupa stack dan heap. Xinu mencatat free memory block menggunakan linked list melalui variabel memlist. Command freememory berhasil dibuat untuk menampilkan daftar free memory block dan menghitung total free space berdasarkan seluruh block yang tersedia.

DAFTAR PUSTAKA

1. Modul 11 Memori Xinu, Praktikum Sistem Operasi, Telkom University Purwokerto.
2. Jurnal Praktikum Sistem Operasi Modul 11: Memori Xinu, Telkom University Purwokerto.